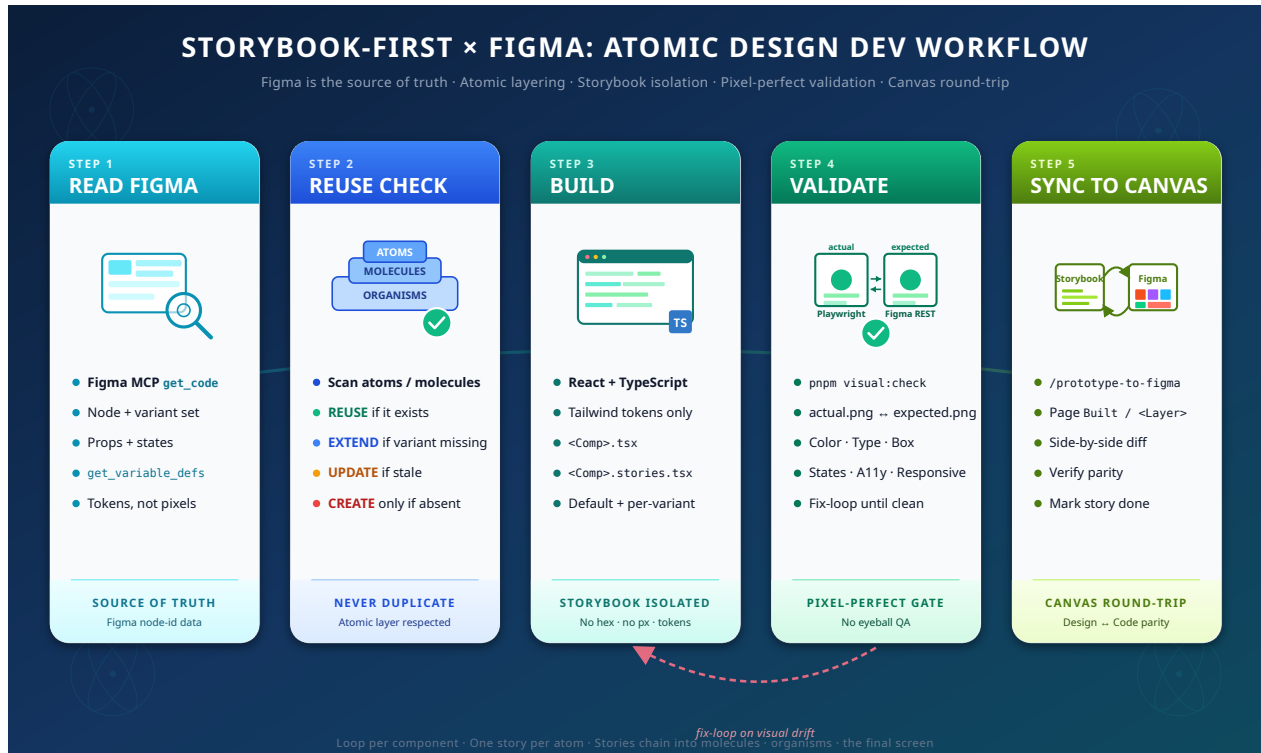


Storybook-First with Figma

A bidirectional workflow for pixel-perfect atomic design



Problem

Two failure modes dominate UI work on real products.

Visual approximation. A button ends up with a hex that is *close enough* to the design. Spacing snaps to Tailwind's default scale instead of the exact Figma value. The font looks right but renders as Inter where Barlow Semi Condensed was specified. None of these get caught in code review — they surface when a stakeholder opens the page next to the Figma file and notices that nothing quite matches.

Silent duplication. Every developer builds their own button, their own badge, their own way of truncating text. Six months in, the project has three `<Button>` components and four different definitions of "primary blue."

Storybook-First with Figma is a workflow that closes both gaps. It runs inside a BMad-driven dev agent, bound to a five-step loop that ends with an automated screenshot diff against the Figma source.

The solution

Every UI story follows the same five-step per-component loop, prefaced by a Step 0 that runs once per story to handle decomposition and planning.

#	Step	What happens
0	Refine the story	Resolve the Figma link; classify the target as atom / molecule / organism / screen; decompose recursively to find every lower-layer dependency; run a Reuse Check on each node (REUSE / EXTEND / UPDATE / OFF-LAYER / CREATE); produce a bottom-up build plan.
1	Read Figma	<code>get_code</code> + <code>get_variable_defs</code> via Figma MCP. Tokens come back named, never as raw hex.
2	Reuse Check	For each component the build plan touches: import if it exists, extend if a variant is missing, HALT if there's a design conflict or a brownfield off-layer match.
3	Build	React + TypeScript + Tailwind. Tokens only — never hardcoded hex/px. <code><Component>.tsx</code> + <code><Component>.stories.tsx</code> per Figma variant.
4	Validate	<code>pnpm visual:check</code> runs Playwright at 2x DPR on the Storybook story → <code>actual.png</code> ; the Figma REST API renders the source node at 2x → <code>expected.png</code> . The agent reads both PNGs and walks the Visual Checklist (color, typography, box, states, a11y). Drift triggers a fix loop.
5	Sync to Canvas	<code>/prototype-to-figma</code> (a Figma MCP skill) pushes the built component to a <code>Built / <Layer></code> page in the same Figma file. Designers verify parity side-by-side with the source frame.

The 5-step loop is per-component. Step 0 decides which loops will run, and in what order — atoms first, then their consumers, finally the original target.

How it works

The module is a thin **execution discipline layer** on top of standard BMad. There is no fork, no patched skill — only one customization file and one helper script:

- `_bmad/custom/bmad-dev-story.toml` holds the policy. When the user invokes `/bmad-dev-story` in Claude Code (VS Code extension), BMad's customization resolver loads every entry of this file's `persistent_facts` array as foundational context for the run. That is where the 5-step loop, the visual-check gate, the token-only rule, the layer enforcement, and Step 0 live. The agent is bound by these rules regardless of who authored the story.
- `docs/workflows/storybook-first-with-figma.md` is the operating manual, loaded as a `file:` reference inside `persistent_facts` . It contains the per-step detail and the Visual

Checklist.

- `scripts/visual-check.mjs` is a ~70-line script. Playwright screenshots the Storybook story at 2x DPR; the Figma REST API renders the source node at 2x. Both files land in `_visual-checks/<component>/<variant>/`. The agent (multimodal) reads both and walks the checklist.

Adoption reality. Most teams install this into projects that have been running for years. The first stories the team points it at will frequently target an organism or a screen with no atoms in `src/components/atoms/` yet. Step 0 handles this by decomposing the target recursively and building bottom-up in a single dev-story run — atoms first, then molecules, then organisms, then the target. Each component still passes through the 5-step loop and visual validation. The agent only HALTs in three cases: an **UPDATE conflict** (component exists at the right layer but differs from Figma), an **OFF-LAYER find** (component exists in the codebase but outside the atomic folder structure — common in brownfield), or an **explicit scope error** in the original story ("build all the molecules" — should be split via a discovery pattern).

The Figma file matters. The workflow's value is bounded by what's in the file. Variables for design tokens, atomic-design organization (frames named or grouped as Atoms / Molecules / Organisms / Screen), and Components with complete variant sets. Without them, tokens get derived from inline values, classification becomes guesswork, and Step 0 produces a noisier build plan.

Install and use

Open a terminal **inside the project root** you want to add the module to (the integrated terminal in VS Code works — files land in your working tree and show up in the Source Control panel for review). Run:

```
bash <(curl -fsSL https://raw.githubusercontent.com/keith-sarate/story-book-first-with-figma/main/install.sh)
```

The script fetches its templates, prompts for your Figma file key and the four Atomic-Design node-ids, drops 15 files into the current directory, and prints next steps. Review the diff in VS Code; commit when you are happy.

After install, three things happen once:

1. In Claude Code (VS Code extension), invoke `/mcp` and approve the `figma` server.
2. `cp .env.example .env` and paste a Figma personal access token from figma.com/settings.
3. Open Claude Code and invoke `/bmad-dev-story` with the path to the story you want to run — for example, a story your PM created from a PRD with a Figma link in its References

section.

The dev agent picks up the story, executes Step 0 (refinement + recursive build plan), confirms with you if more than three components will be created, then runs the 5-step loop per component, bottom-up. Every component lands in the correct layer folder, every Storybook story passes the Visual Checklist, every result is pushed back to Figma on a `Built / <Layer>` page.

Repository: github.com/keith-sarate/story-book-first-with-figma · **License:** MIT